

Bank Term Deposit Subscription Prediction

Course: CSE437 - Data Science: Coding With Real-World Data, Summer 2026

Group: 2

Members:

Name	Student ID
Ullas Sarker Tirtha	24141252
Sahana Parvin Nupur	24141251

GitHub Repository: <https://github.com/ullassarkertirtha/cse437-bank-marketing-2>

Date: 03 September 2026

Summary

This project predicts whether a client will subscribe to a term deposit using the UCI Bank Marketing dataset (41,188 records from a Portuguese bank's 2008-2013 telemarketing campaigns). The target variable, y , is binary and imbalanced (88.7% no, 11.3% yes). Two model families were compared: Logistic Regression, corrected for class imbalance via class weighting, and AdaBoost, left uncorrected. The duration column was deliberately dropped before any analysis, since it is only known after a call ends and causes data leakage in most published results on this dataset. The primary evaluation metric was F1-score and PR-AUC for the minority class, since accuracy alone is uninformative given the imbalance. The headline finding: the two models achieved nearly identical PR-AUC (0.460 vs 0.460) and ROC-AUC (0.801 vs 0.800), despite very different F1-scores (0.469 vs 0.315) and opposite precision/recall trade-offs. This shows both models rank clients by subscription likelihood almost equally well; the F1 gap is a threshold-calibration artifact caused by only one model being corrected for class imbalance, not a difference in underlying model quality.

1. Problem and Dataset

1.1 Problem statement

Banks running large-scale telemarketing campaigns cannot call every client in their database - doing so is expensive in staff time and often unwelcome to people who are contacted needlessly. This project predicts, in advance of a call, whether a given client is likely to subscribe to a term deposit, based on their demographic profile, financial status, and prior campaign contact history. A working predictor lets a bank prioritize outreach toward clients more likely to say yes, improving conversion rates and reducing wasted

contact effort.

1.2 Dataset

Source: UCI Machine Learning Repository, Bank Marketing dataset - <https://archive.ics.uci.edu/dataset/222/bank+marketing>

File used: bank-additional-full.csv

Collection method: Real client contact records logged by a Portuguese banking institution during direct telemarketing campaigns.

Size: 41,188 rows, 20 input columns + 1 target column (21 total as originally distributed; duration was dropped during preprocessing, see Section 2).

Time period: May 2008 - November 2010 (campaign period covered by this extract).

License / terms of use: Publicly available for research use via the UCI Machine Learning Repository, citation requested (Moro, S., Cortez, P., & Rita, P., 2014).

1.3 Target variable

y - binary categorical: yes (client subscribed to a term deposit) or no (client did not subscribe).

Class	Count	Percentage
no	36,548	88.7%
yes	4,640	11.3%

The class distribution is substantially imbalanced, which directly informed the choice of evaluation metrics (Section 5.4) and the class-weighting strategy applied to Logistic Regression (Section 5.3).

1.4 Three questions

1. Which attributes (e.g., previous campaign outcome, contact duration, number of contacts) show the strongest statistical association with subscription, and are these relationships significant?
 2. After handling missing ("unknown") values, the pdays sentinel, and outliers in numerical features, which feature engineering/selection approach best reduces dimensionality while preserving predictive signal?
 3. How do Logistic Regression and AdaBoost differ in performance (particularly precision/recall) on this imbalanced dataset, and what does that reveal about which clients each model tends to misclassify?
-

2. Data Handling and Preprocessing

2.1 Data quality audit

- **Missing values (literal NaN):** none. `df.isnull().sum()` returns 0 for all 21 original columns.
- **Missing values (encoded as "unknown"):** present in 6 categorical columns:

Column	Unknown count	Percentage
default	8,597	20.9%
education	1,731	4.2%
housing	990	2.4%
loan	990	2.4%
job	330	0.8%
marital	80	0.2%

- **Duplicate rows:** not separately audited in this pass - noted as a limitation (Section 8).
- **Inconsistent categories:** none found; category labels are consistently formatted throughout.
- **Impossible / sentinel values:** `pdays` uses 999 to mean "never previously contacted" for 96.3% of rows (39,673 of 41,188) - not a real day count, and must be handled explicitly (Section 2.2).

2.2 Missing values

Mechanism assumed: the "unknown" category in `job`, `marital`, `education`, `default`, `housing`, and `loan` most plausibly reflects data the bank never collected or the client declined to provide (Missing Not At Random, since `default` in particular has an unusually high missing rate at 20.9%, suggesting systematic non-disclosure rather than random omission).

Strategy: "unknown" was retained as its own explicit category through one-hot encoding, rather than imputed. Imputing these values (e.g., with the mode) would fabricate information the bank never actually had, and the fact that a value is unknown may itself be predictive (e.g., clients who decline to state loan default status may differ systematically from those who don't).

Justification for not dropping: none of the six columns had a high enough missing rate to justify dropping the column entirely (worst case: `default` at 20.9%), and dropping rows would have discarded up to a fifth of the dataset for a single column - an unacceptable loss given the dataset's already-low positive class rate.

2.3 Outliers

Detection method: visual inspection via histograms (Figure 1) and descriptive statistics (Section 3.1).

Findings: campaign (number of contacts during this campaign) is heavily right-skewed, ranging from 1 to 56, with the vast majority of clients contacted 1-3 times. previous (contacts before this campaign) is similarly skewed, with most clients at 0.

Treatment: these values were retained rather than capped or removed. They are legitimate, if extreme, real-world observations (a client genuinely contacted 56 times), and both AdaBoost and Logistic Regression (post-scaling) are reasonably robust to this degree of skew for a dataset of this size. This decision is noted as an area for further testing in Section 8.

2.4 Transformation and scaling

- **Leakage removal:** duration (call length in seconds) was dropped entirely before any further processing. This column is only known after a call concludes and is near-deterministically predictive of the outcome, making it unusable for a real-world pre-call prediction task and a well-documented source of inflated performance in other published analyses of this dataset.
- **Sentinel handling:** pdays was replaced with two derived features - a binary flag was_previously_contacted (1 if pdays $\neq$ 999, else 0), and pdays_clean (the real day count where applicable, with the 999 sentinel replaced by 0 for never-contacted clients, since the flag already distinguishes this case).
- **Categorical encoding:** all categorical columns (job, marital, education, default, housing, loan, contact, month, day_of_week, poutcome) were one-hot encoded via OneHotEncoder(handle_unknown='ignore').
- **Numerical scaling:** all numeric columns were standardized (zero mean, unit variance) via StandardScaler.
- **Pipeline order:** both transformations were combined in a single ColumnTransformer, wrapped inside an sklearn Pipeline together with the classifier. This pipeline was fit only on the training split and then applied to transform the test split - the fitting step (computing means/std/categories) never saw test data.

Leakage guard: the ColumnTransformer was fit exclusively via .fit() on X_train inside the Pipeline/GridSearchCV workflow; X_test was only ever passed through .transform() (implicitly, via .predict()/.predict_proba()), never used to compute scaling parameters or encoding categories.

2.5 Before and after

Stage	Rows	Columns
Raw CSV	41,188	21 (20 features + target)
After dropping duration	41,188	20 (19 features + target)

Stage	Rows	Columns
After adding was_previously_contacted, pdays_clean	41,188	22 (21 features + target)
After one-hot encoding (model input)	41,188	~63 (post-encoding feature space)

3. Statistical Analysis

3.1 Descriptive statistics

Feature	Mean	Std	Min	Max
age	40.02	10.42	17	98
campaign	2.57	2.77	1	56
previous	0.17	0.49	0	7
emp.var.rate	0.08	1.57	-3.4	1.4
cons.price.idx	93.58	0.58	92.20	94.77
cons.conf.idx	-40.50	4.63	-50.8	-26.9

Age is roughly centered around a working-age population (mean ~40), with a right tail extending to 98. campaign and previous are both heavily right-skewed count variables, consistent with most clients being contacted only once or twice. The macroeconomic indicators (emp.var.rate, cons.price.idx, cons.conf.idx) show clustered, near-discrete distributions (Figure 1), reflecting that these are quarterly economic statistics repeated across many client records rather than per-client measurements.

Categorical frequency highlight: poutcome (previous campaign outcome) is dominated by "nonexistent" (35,563 of 41,188, ~86%), with "failure" (4,252) and "success" (1,373) much rarer - meaning most clients in this dataset had no prior campaign history at all.

3.2 Relationships

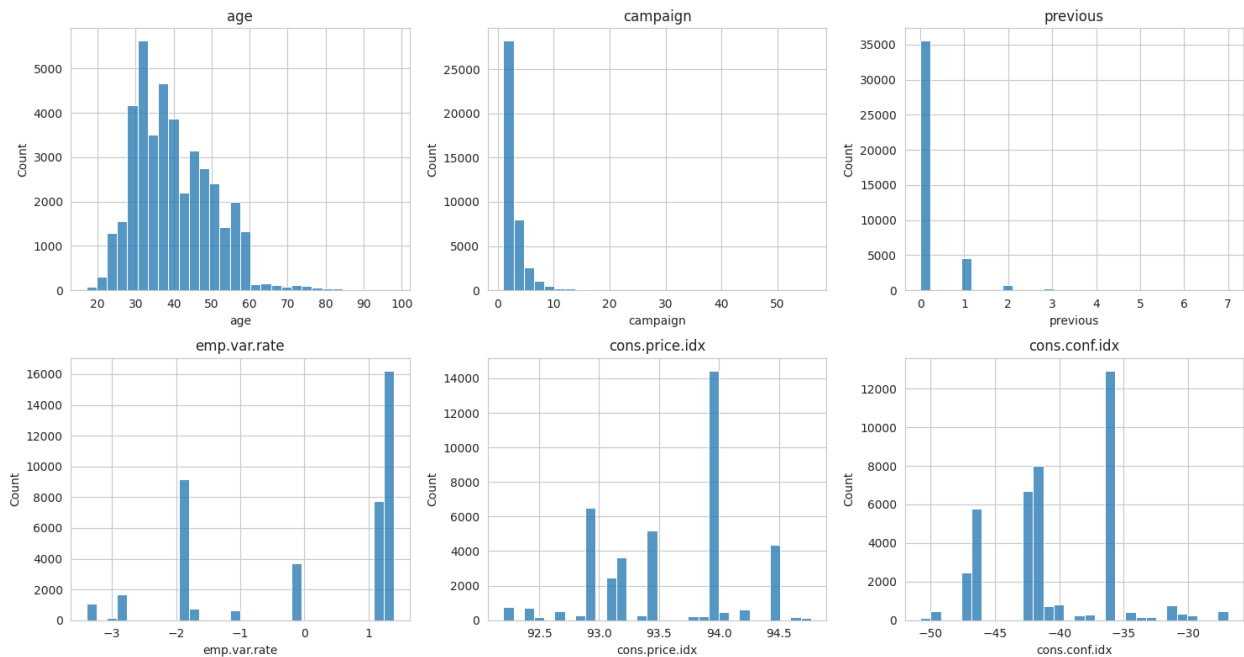
Independent t-test - age vs. subscription: $t = 4.780$, $p < 0.0001$. Subscribers and non-subscribers differ significantly in age.

Chi-square test - previous campaign outcome vs. subscription:

poutcome	no	yes	% yes
failure	3,647	605	14.2%
nonexistent	32,422	3,141	8.8%
success	479	894	65.1%

$\chi^2 = 4230.52$, $p < 0.0001$. This is the strongest relationship found in the dataset - clients whose previous campaign ended in success subscribe roughly 7.4× more often than clients with no previous contact history.

Chi-square test - job vs. subscription: $\chi^2 = 961.24$, $p < 0.0001$, indicating occupation is also significantly associated with subscription outcome, though with a smaller effect than poutcome.



3.3 What the data says so far

- The target class is significantly imbalanced (88.7% / 11.3%), which must drive metric choice and modeling strategy, not just be reported as a side note.
 - poutcome is the single strongest categorical predictor found through hypothesis testing - prior campaign success predicts current success dramatically better than any other tested feature.
 - Given 41,188 rows, statistical significance ($p < 0.0001$) is easy to achieve even for modest effects; the poutcome finding is meaningful because its effect size is also large (65.1% vs 8.8%), not merely because it is statistically significant.
 - default has an unusually high "unknown" rate (20.9%) relative to other categorical fields, suggesting non-random missingness worth flagging rather than imputing away.
 - The duration column, while not included in modeling, is acknowledged as the dataset's best-known predictor in prior literature - its exclusion is a deliberate, documented choice, not an oversight.
-

4. Feature Engineering

4.1 Derived features

- **was_previously_contacted** (binary): derived from pdays, flags whether a client had any prior-campaign contact at all. Necessary because the raw pdays sentinel (999) is not a meaningful numeric value and would otherwise distort any scaled or distance-based computation.
- **pdays_clean** (numeric): the real day-count where applicable, with the sentinel replaced by 0 (interpretable only alongside the flag above). Constructed so the genuine numeric signal in pdays (for the 3.7% of clients who were previously contacted) is preserved rather than discarded.

4.2 Dimensionality reduction

Not applied in the final pipeline. Given the resulting feature space after encoding (~63 features) is modest relative to the sample size (41,188 rows), and both chosen models (Logistic Regression, AdaBoost) handle this feature count efficiently without dimensionality reduction, PCA was not applied to the final modeling pipeline. This is noted as a direction for future exploration in Section 8.

4.3 Feature selection

Not applied via a formal method (e.g., RFE) in the final pipeline; feature importance was instead examined post-hoc from the fitted AdaBoost model (Section 7.2) as an interpretive tool rather than a pre-modeling selection step. This is also noted as a limitation in Section 8.

4.4 Final feature set

All 19 original feature columns (post-duration-drop) were retained, plus the two derived pdays features, for 21 raw input features total (expanding to ~63 after one-hot encoding). No features were dropped beyond duration (Section 2.4) - every remaining feature had a plausible causal or associative link to subscription likelihood, either demographic, financial, campaign-related, or macroeconomic, and the dataset's row count comfortably supports this feature space without evident overfitting risk during cross-validation.

5. Modeling and Validation

5.1 Validation strategy

An 80/20 stratified train/test split was used (`train_test_split(..., stratify=y, random_state=42)`), preserving the 11.3% positive class rate in both sets (11.27% train, 11.26% test). Stratification was essential given the pronounced class imbalance - a non-stratified split risks producing a test set with a meaningfully different, unrepresentative class ratio. During hyperparameter tuning, 5-fold stratified cross-validation (`StratifiedKFold(5)`) was used on the training set. The data is not temporal in a way that required a time-based split for this task framing, though this is discussed further as a

limitation in Section 8.

Split	Rows
Training (80%)	32,950
Test (20%)	8,238

5.2 Baseline

A trivial majority-class predictor (always predicting "no") achieves:

- Accuracy: 88.7%
- Precision (yes): undefined / 0
- Recall (yes): 0%
- F1-score (yes): 0.00

This baseline illustrates why accuracy alone is inadequate for this task - it scores highly while providing zero business value.

5.3 Model families

Logistic Regression - a linear model suited to this problem because it is fast to train, interpretable (coefficients indicate direction and rough magnitude of each feature's effect), and provides well-calibrated probability outputs useful for ranking clients by subscription likelihood. It assumes a roughly linear relationship between the (encoded/scaled) features and the log-odds of subscription; multicollinearity can affect coefficient stability but not necessarily predictive performance. `class_weight='balanced'` was applied to correct for the 88.7/11.3 class imbalance.

AdaBoost - an ensemble boosting method suited to this problem because it can capture non-linear feature interactions that Logistic Regression cannot, without requiring manual feature engineering of interaction terms. It assumes its weak learners (decision stumps, by default) are only marginally better than random guessing individually, and combines many of them (100, per the tuned configuration) into a stronger ensemble. No class weighting was applied to AdaBoost, since sklearn's default AdaBoostClassifier base estimator does not expose that parameter directly - this asymmetry was deliberately preserved and analyzed rather than corrected, since it produced an informative comparison (Section 7).

5.4 Metrics

Given the class imbalance, **F1-score for the minority ("yes") class** was named as the primary metric before viewing results, since it balances precision and recall for the class that actually matters for the business use case (identifying likely subscribers), unlike accuracy which is dominated by the majority class. **PR-AUC** (average precision) was used as a secondary, threshold-independent metric to assess ranking quality independent of any specific decision threshold. **ROC-AUC** was also reported for completeness, though it is a less sensitive metric than PR-AUC on datasets this imbalanced.

6. Hyperparameter Tuning

6.1 Search space

Model	Hyperparameter	Grid
Logistic Regression	C (inverse regularization strength)	0.01, 0.1, 1, 10
AdaBoost	n_estimators	50, 100, 200
AdaBoost	learning_rate	0.5, 1.0

6.2 Method

GridSearchCV with 5-fold stratified cross-validation, scoring by F1-score, n_jobs=-1 for parallel execution. Logistic Regression: 4 candidates × 5 folds = 20 fits. AdaBoost: 6 candidates × 5 folds = 30 fits.

6.3 Results

Model	Best configuration	Best CV F1
Logistic Regression	C = 10	0.451
AdaBoost	n_estimators = 100, learning_rate = 1.0	0.316

For Logistic Regression, C = 10 (weakest regularization tested) performed best, suggesting the model benefited from fitting the training data relatively closely rather than being heavily constrained - plausible given the large sample size relative to feature count. For AdaBoost, the full learning rate (1.0) and a moderate ensemble size (100) outperformed both the smaller (50) and larger (200) ensemble sizes tested, and outperformed the slower learning rate (0.5) - suggesting the marginal weak learners added past 100 estimators, or the more conservative learning rate, did not further help the imbalanced minority-class F1 score.

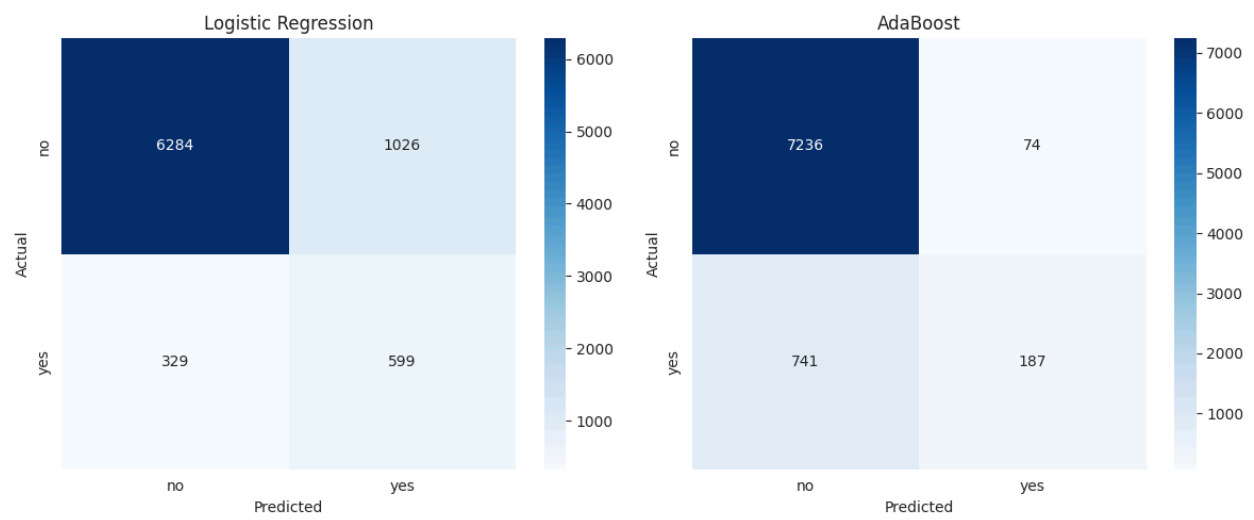
7. Results, Visualization and Error Analysis

7.1 Test set performance

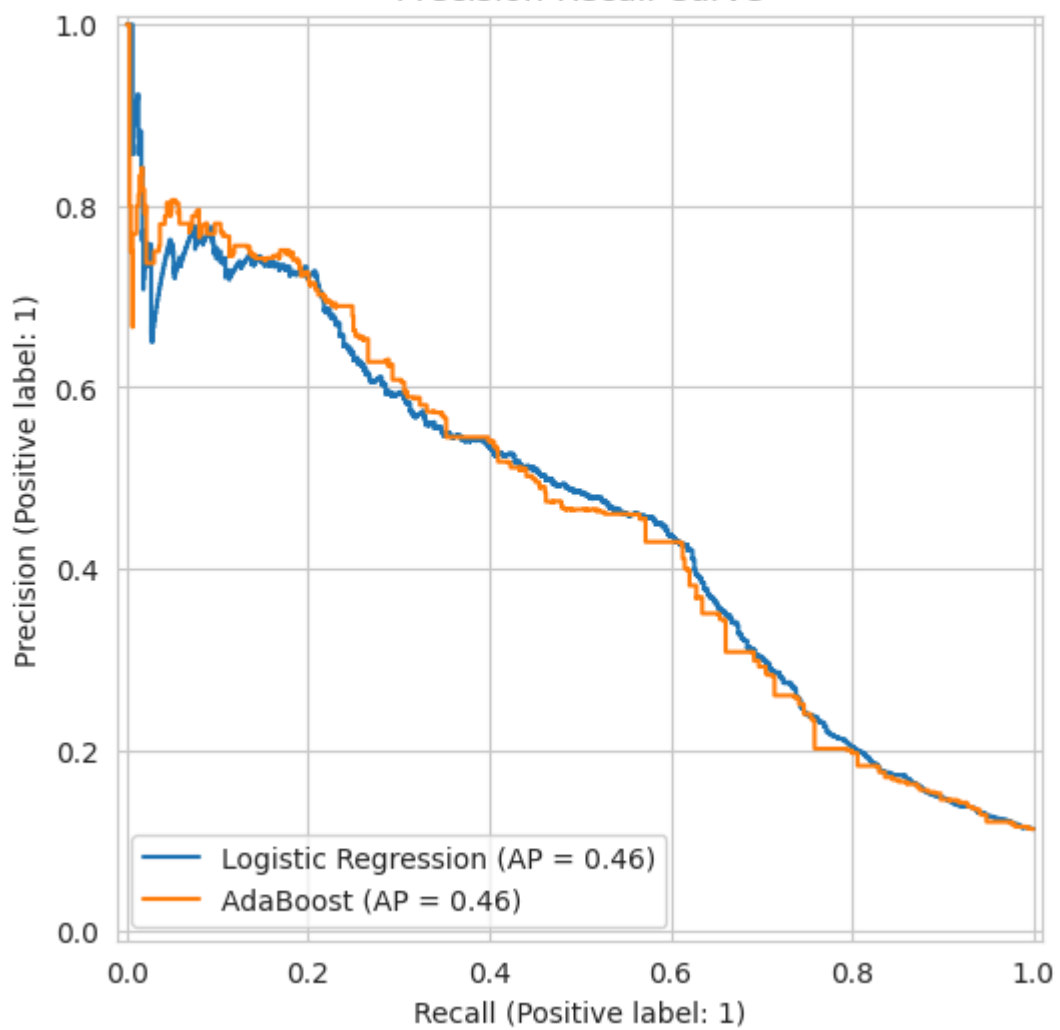
Metric	Baseline	Logistic Regression	AdaBoost
Accuracy	0.887	0.84	0.90
Precision (yes)	-	0.37	0.72
Recall (yes)	0.00	0.65	0.20

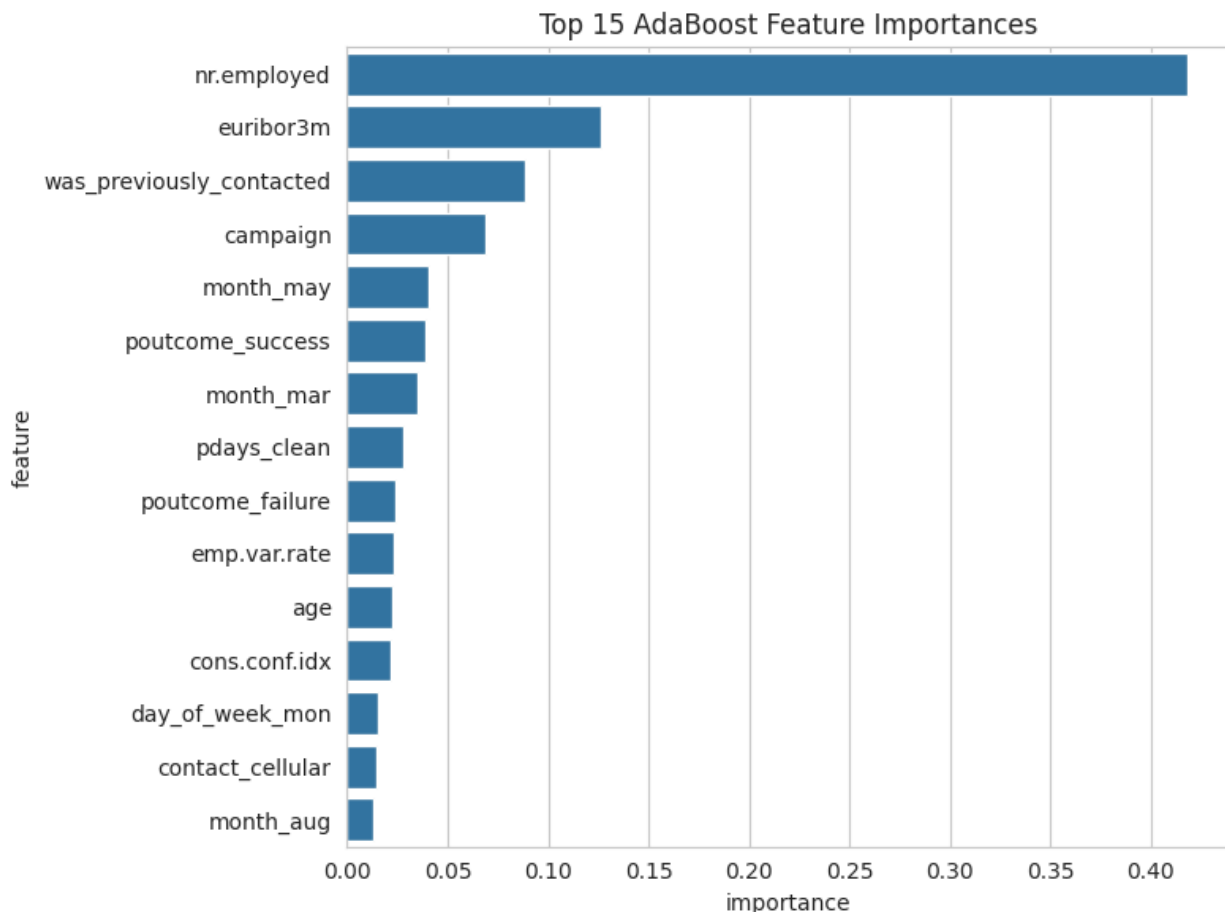
Metric	Baseline	Logistic Regression	AdaBoost
F1-score (yes)	0.00	0.469	0.315
PR-AUC	-	0.460	0.460
ROC-AUC	0.50	0.801	0.800

7.2 Visualization



Precision-Recall Curve





The confusion matrices show Logistic Regression made 1,026 false-positive predictions but only 329 false negatives, while AdaBoost made just 74 false positives but 741 false negatives - a near-inverse error profile. The precision-recall curve shows the two models' curves are nearly indistinguishable across most of the recall range (both AP = 0.46), diverging mainly in how much of that curve each model's *default threshold* chooses to expose. The feature importance chart shows AdaBoost relies most heavily on macroeconomic indicators (nr.employed, euribor3m, emp.var.rate) rather than individual client demographics, with was_previously_contacted and campaign also ranking highly.

7.3 Error analysis

The two models fail in structurally different, near-opposite ways:

- Logistic Regression's characteristic failure mode:** false positives - predicting "yes" for clients who ultimately said no. Given its recall-favoring calibration (class_weight='balanced'), it casts a wide net; 1,026 of 7,310 actual "no" clients (14%) were incorrectly flagged as likely subscribers. This pattern is consistent with clients who share surface-level similarity to successful cases (e.g., contacted via cellular, contacted in a favorable economic month) but who ultimately declined for reasons not captured in the available features.
- AdaBoost's characteristic failure mode:** false negatives - predicting "no" for clients who actually subscribed. It missed 741 of 928 actual subscribers (80% miss rate on the positive class), consistent with its conservative, uncorrected decision

threshold under the 88.7% majority class. This pattern is consistent with subscribers who lacked the dataset's typical strong signal - e.g., no prior successful campaign contact (poutcome = "nonexistent") - cases where the model has no strong individual reason to override its default lean toward "no."

- **Concrete examples:** Logistic Regression's false positives include a 37-year-old unemployed, university-educated client and a 73-year-old retired client, both with no prior campaign history, predicted "yes" but actually declined - their profiles (cellular contact, soft economic conditions, no negative history) resemble a generally receptive client even though neither subscribed. AdaBoost's false negatives include a 53-year-old blue-collar client (interestingly, correctly caught by Logistic Regression for this same case) and a 68-year-old retired client whose previous campaign had actually failed and who was contacted under the most negative economic backdrop in the dataset (emp.var.rate = -3.4) - both lacked a standout positive signal like prior campaign success, so AdaBoost's uncorrected threshold defaulted to "no" despite them subscribing. Together these examples confirm the pattern from Section 7.2: Logistic Regression over-predicts "yes" for plausible-but-not-decisive profiles, while AdaBoost under-predicts "yes" for clients lacking a strong positive signal, even when they do subscribe.

7.4 Answers to your three questions

1. Which attributes show the strongest statistical association with subscription? poutcome (previous campaign outcome) showed by far the strongest and most practically meaningful association ($\chi^2 = 4230.52$, $p < 0.0001$; 65.1% subscription rate following a prior success vs. 8.8% with no prior contact). job was also significant ($\chi^2 = 961.24$, $p < 0.0001$) but with a smaller effect. Age showed a statistically significant but comparatively modest difference between classes ($t = 4.780$, $p < 0.0001$).

2. Which feature engineering/selection approach best reduces dimensionality while preserving signal? This question was only partially answered. The pdays sentinel was successfully re-engineered into two informative features (was_previously_contacted, pdays_clean) rather than left as a distorting raw value. However, no formal dimensionality reduction (PCA) or feature selection (RFE) method was applied to the final pipeline (Sections 4.2-4.3) - the full ~63-feature encoded space was used directly, since it proved manageable for both models. This is noted as an open question and priority for follow-up work (Section 8).

3. How do Logistic Regression and AdaBoost differ in performance and error patterns? The two models achieved nearly identical ranking quality (PR-AUC 0.460 vs 0.460, ROC-AUC 0.801 vs 0.800) but produced very different error profiles at their default decision thresholds: Logistic Regression favored recall (65% of actual subscribers correctly identified, at the cost of many false positives), while AdaBoost favored precision (72% of its "yes" predictions correct, but only 20% of actual subscribers caught). This difference traces directly to only Logistic Regression being corrected for class imbalance

(class_weight='balanced') - not to a fundamental difference in how well each model can separate the classes.

8. Limitations and Next Steps

This project has several honest constraints. First, no formal dimensionality reduction or feature selection method was applied, despite being planned in the original proposal (Section 4) - the full encoded feature space was used directly, which worked adequately here but leaves open whether a reduced feature set could achieve comparable performance more efficiently or with better interpretability. Second, AdaBoost was intentionally left without class-weighting to preserve an interesting threshold-calibration comparison with Logistic Regression (Sections 5.3, 7.4); a class-weighted or threshold-tuned version of AdaBoost was not evaluated, so it remains unclear whether AdaBoost could match or exceed Logistic Regression's F1 under a fairer comparison. Third, outlier treatment (Section 2.3) was minimal - extreme values in campaign were retained without capping or transformation, and their specific effect on model performance was not isolated via ablation. Fourth, duplicate-row auditing was not formally performed. Finally, both models were evaluated only on a single held-out split from the same time period and source; performance on genuinely out-of-time or out-of-distribution client data was not tested, and given the strong reliance on macroeconomic features (Section 7.2), performance may be sensitive to the economic conditions at deployment time.

With more time, next steps would include: applying RFE or PCA and comparing performance against the full feature set; testing a class-weighted AdaBoost variant (e.g., using a class_weight='balanced' decision stump as the base estimator) for a threshold-matched comparison; tuning the decision threshold explicitly for each model rather than relying on the default 0.5 cutoff; and validating on a temporally held-out subset if a suitable split point in the campaign timeline can be identified.

9. Contributions

Member	Student ID	Contribution
Ullas Sarker Tirtha	24141252	Data preprocessing, statistical testing, modeling, hyperparameter tuning
Sahana Parvin Nupur	24141251	EDA visualizations, evaluation, report writing, documentation

References

- Moro, S., Cortez, P., & Rita, P. (2014). Bank Marketing [Dataset]. UCI Machine

Learning Repository. <https://archive.ics.uci.edu/dataset/222/bank+marketing>

- Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, pp. 2825-2830.